Pointer-First Agent Constitution for Stela: Architectural Certification and Skill System Refactoring

1. The Crisis of Context and the Imperative for Deterministic State Machines

The evolution of repository-resident Artificial Intelligence (AI) has precipitated a fundamental crisis in software governance: the irreconcilable tension between the probabilistic fluidity of generative logic and the rigid, binary requirements of structural integrity. In the Stela ecosystem, this tension is not merely a theoretical concern but an operational bottleneck that threatens the viability of the platform. The "Operator" (human authority) and the "Integrator" (AI lead) operate within a high-entropy environment where the slightest divergence in instruction sets—a phenomenon known as "Drift"—can cascade into systemic failure.[1]

The prevailing methodology of embedding verbose, repetitive, and static instructions directly into task definitions has rendered the current agentic system effectively "un-serviceable." This legacy approach relies on "thick" context windows, where prompts are loaded with paragraphs of behavioral instructions, coding guidelines, and narrative project history. While intuitive, this approach suffers from a critical flaw identified in recent research on context engineering: as the context window expands with redundant text, the "Signal-to-Noise" (SNR) ratio diminishes, leading to a degradation in the agent's ability to retrieve and apply the correct constraints.[1]

In the Stela environment, this manifests as "Ghost Canon"—outdated rules lingering in the text of old tasks or agent definitions that compete with the active governance layers. For instance, if an agent is instructed via natural language to "maintain clean code," it relies on its internal training data (probabilistic) to interpret "clean," which may conflict with the specific, immutable standards defined in the repository's PoT.md (Policy of Truth). This creates a "Split-Brain" scenario where the Agent follows a hallucinated or outdated standard while the Integrator attempts to enforce the current law.[1]

To resolve this crisis, Stela must undergo a radical architectural transformation toward a "Pointer-First Agent Constitution." This paradigm shift moves the repository from a text-heavy, instruction-based model to a sparse, reference-based architecture. Under this constitution, no canonical text is duplicated. Instead, every operational entity—whether an Agent, a Task, or a Skill—functions as a pointer to an immutable "Single Source of Truth" (SSOT) defined in the Policy of Truth.

1.1 The Theoretical Basis of Context Drift

Drift in AI agent systems is defined as the gradual divergence between the system's intended state, as defined by its governance documentation, and its actual operational state. In legacy architectures, the reliance on natural language instructions creates a vulnerability to semantic interpretation. For example, an instruction to "review code for errors" is subjective. Does "error" include stylistic deviations? Does it include security vulnerabilities? The agent, relying on its training data, may interpret this differently from session to session, introducing non-deterministic behavior into the repository.[1]

The Pointer-First Agent Constitution addresses this by effectively eliminating the "instruction" layer from the agent's context. Instead of telling an agent *how* to behave (which is prone to interpretation), the architecture provides the agent with a *pointer* to an executable tool that enforces the behavior. This shifts the governance model from "requesting compliance" (probabilistic) to "preventing violation" (deterministic). By reducing the prompt to a sparse graph of pointers, we maximize the SNR, ensuring that the agent's cognitive resources are dedicated solely to payload execution rather than interpreting ambiguous governance text.[1]

1.2 The "Un-Serviceable" State of Legacy Artifacts

A forensic audit of the provided repository dump reveals a system in a state of transition. While the directory structure (ops/, docs/, projects/) suggests a mature governance model, the internal composition of the agent files (R-AGENT-01 through R-AGENT-06) and task definitions (B-TASK-01 through B-TASK-10) remains rooted in legacy, narrative-driven paradigms. These artifacts are currently "un-serviceable" because they lack Referential Integrity—the guarantee that every instruction resolves to a valid, existing, and immutable source of truth.[1]

Legacy tasks often duplicate logic that should reside in the ops/ binary layer. For example, a task might include a multi-step instruction to "run git status, check for changes, and commit," rather than pointing to a unified ops/bin/open or ops/lib/scripts/git-prep.sh script. This redundancy creates a maintenance nightmare: if the git workflow changes, every single task file must be updated manually. If one is missed, the system enters a state of inconsistency. Furthermore, the agents themselves are defined by what they "are" (e.g., "I am a code reviewer") rather than what they "execute" (e.g., "Run tools/lint/style.sh"). This reliance on persona over process makes them susceptible to "hallucinations," where the agent invents new procedures that sound plausible but violate the strict "Filing Doctrine" of the repository.[1]

---

2. Constitutional Foundations: The Policy of Truth (PoT) as Physical Law

The stability of the Stela platform rests entirely upon the "Policy of Truth" (PoT.md), which

functions not merely as a guideline but as the immutable physical law of the repository. Any architectural proposal that does not treat PoT.md as the supreme antecedent to all other logic is functionally invalid. The PoT.md file explicitly articulates a doctrine of preventative governance: "The system prevents action that violates policy. It does not request compliance".[1]

2.1 The Staffing Protocol: A Tripartite Cognitive Division

The governance model defined in PoT.md establishes a rigid hierarchy of authority and cognitive function known as the Staffing Protocol. This is not merely a user role definition; it is a structural constraint on the types of actions permitted by different entities within the system.

- **Operator (Human):** The Operator is the sole source of finality. They own final decisions, approvals, and secrets. Crucially, the Operator "performs all commits, pushes, and merges".[1] This constraint creates a hard boundary for the AI agents: no autonomous agent is permitted to write directly to the repository history. A task that includes instructions like "Commit the changes" or "Push to main" is structurally invalid because it commands the AI to perform an action reserved for the Operator. In the Pointer-First system, tasks must be written to produce artifacts (diffs, patches, shell scripts) that the Operator can review and execute.
- **Integrator (Lead AI):** The Integrator "maintains governance, structural integrity, and auditing".[1] It generates Dispatch Packets and detects "Drift." This persona operates at the meta-level, functioning as the "immune system" of the repository. The tasks assigned to an Integrator (e.g., B-TASK-04 Plan Command or B-TASK-08 Draft DP) must focus on the manipulation of the governance surface (TASK.md, SoP.md) rather than the project surface. The Integrator relies on the ops/bin/dump tool to perceive the system state; without this tool, the Integrator is effectively blind.[1]
- **Contractor (Guest AI):** The Contractor "executes specific logic tasks and drafts implementation details within a defined scope".[1] This is the execution arm of the system. The Contractor has the highest risk of introducing entropy (bad code, typos, contractions) and therefore operates under the strictest constraints. The Contractor does not need to "know" the security policy; it needs to know *where* the policy is and be compelled to check its work against it via specific S-LEARN modules.

2.2 The Behavioral Logic Standard: Encoding Discipline

The Behavioral Logic Standard defines the specific operational parameters that all AI agents must adhere to. These are not abstract ideals; in a Pointer-First architecture, they must be encoded into binary gates that enforce compliance mechanically.

Linguistic Precision and Rhythmic Integrity

The standard mandates "Linguistic Precision," specifically highlighting the "Contraction Prohibition." Agents must refrain from using contractions (e.g., "don't," "can't") to ensure

rhythmic precision for technical auditing. In a legacy system, this would be a text instruction: "Do not use contractions." This invites failure because "Don't" is itself a contraction, creating a semantic paradox for the LLM. In the Pointer-First system, this is enforced by tools/lint/style.sh, which uses grep to search for patterns like \b(don't|can't|won't|it's)\b. The B-TASK simply points to the linter. If the Contractor uses a contraction, the linter fails, the build breaks, and the task cannot be marked complete.[1]

Quantitative Reporting and Absolute Literalism

The standard requires "Quantitative Reporting," specifying that agents must "specify exact deviations from established protocols (e.g., 'execution path deviated by 14.7%') rather than using subjective descriptions".[1] Subjective terms like "significant changes" or "minor edits" are forbidden. A serviceable task must point to S-LEARN-06 (Advanced Git Forensics), which contains the specific commands (e.g., git diff --numstat) to generate this data. This aligns with "Protocol Deviation Reporting" standards in rigorous engineering fields, where every variance from the baseline must be quantified.[1]

Furthermore, "Absolute Literalism" requires agents to "interpret instructions with total fidelity" and "seek clarification for any ambiguity before proceeding." This is enforced by the "Freshness Gate" in TASK.md. The Agent must echo: "Loaded: PoT, SoP, CONTEXT, MAP." If the Agent paraphrases this (e.g., "I have loaded the files"), it fails the literalism check, signaling a drift in the agent's cognitive state.[1]

2.3 The Filing Doctrine and Semantic Map

The constitution establishes a rigid semantic separation of concerns through its Filing Doctrine, which must be respected by all B-TASK and R-AGENT entities. This doctrine maps the repository tree to specific cognitive functions, creating a "Semantic Map" that the agent can navigate deterministically.[1]

Directory	Semantic Definition	Cognitive Function	Examples
ops/	**Run**	**Execution:** The "hands" of the system. Contains executable logic, binaries, and automation.	ops/bin/open, ops/bin/dump
docs/	**Explain**	**Memory:** The "cortex" of the	PoT.md, docs/MANUAL.md

		system. Contains manuals, rationale, and governance.	
projects/	**Work**	**Output:** The "payload" where value is generated.	Project source code
storage/	**Trash**	**Ephemeral:** The "scratchpad." Explicitly excluded from canon.	storage/handoff/, storage/dumps/

Legacy tasks that embedded execution logic (which belongs in ops/) or lengthy explanations (which belong in docs/) violated this doctrine. A Pointer-First task respects this separation by containing only the coordination logic necessary to link a specific objective to the correct ops/ tools and docs/ references. It does not "do" the work; it points to the tool that does the work.[1]

2.4 Context Hygiene and Anti-Drift Governance

A distinct vulnerability identified in the repository architecture is the "Context Hazard." PoT.md explicitly defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1] The global context (ops/lib/manifests/CONTEXT.md) is loaded at the start of every session via ops/bin/open. If the library of all possible tasks and skills were included here, two specific failures would occur:

1. **Token Saturation:** The context window would be flooded with irrelevant instructions, degrading the Agent's reasoning capacity for the specific task at hand.
2. **Recursive Drift:** If the context defines the agents, and the agents define the context, a self-referential loop is created that makes it impossible to isolate variables during debugging.

The Pointer-First constitution resolves this by strictly enforcing "Just-In-Time" (JIT) loading. Agents, Tasks, and Skills are never loaded globally. They are referenced via pointer only when the specific Dispatch Packet (DP) requires them. This implies that B-TASK files must be rigorously stripped of inline text; they must remain lightweight enough to be injected into the context on demand without displacing the actual project code.

---

3. Forensic Audit of the Legacy Ecosystem

To define the necessary improvements, we must first conduct a forensic audit of the existing

system state as captured in the repository dump and analysis reports. This audit identifies the specific "un-serviceable" points that the new constitution must rectify.

3.1 Un-Serviceable Points in Legacy Tasks (B-TASK)

The current B-TASK files (B-TASK-01 through B-TASK-10) act as "ghost canon," containing redundant and often conflicting instructions.

- **Logic Redundancy:** Legacy tasks often embed multi-step execution logic that should reside in the ops/ binary layer. For instance, a task might manually instruct an agent to "run git status, check for changes, and commit" instead of pointing to a unified script like ops/bin/open or ops/lib/scripts/git-prep.sh.[1]
- **Persona Over Process:** Tasks are currently defined by what an agent "is" (e.g., "I am a code reviewer") rather than what it "executes" (e.g., "Run tools/lint/style.sh"). This narrative approach makes agents susceptible to "hallucinations" where they invent procedures that violate the repository's "Filing Doctrine".[1]
- **Lack of Binary Enforcement:** Legacy tasks "request compliance" by asking agents to follow rules (e.g., "Don't use contractions") rather than enforcing them via binary gates. This is probabilistic governance, which inevitably leads to drift.[1]

3.2 Un-Serviceable Points in Legacy Agents (R-AGENT)

The legacy agents (R-AGENT-01 through R-AGENT-06) are similarly flawed due to their reliance on narrative descriptions.

- **R-AGENT-01 (Architect):** Defined by "system design" narratives without pointers to docs/MAP.md or ops/bin/map, leading to hallucinated system topologies.
- **R-AGENT-04 (Integrator):** Often attempts to generate Dispatch Packets based on "general understanding" rather than strictly filling the TASK.md template, leading to format drift.
- **R-AGENT-05 (Refactor Cleaner):** Lacks mandatory stop conditions linked to tools/verify.sh, creating a high risk of destructive action where required scaffolding is deleted.[1]

3.3 The Failure of the Skill System (S-LEARN)

The skill system (S-LEARN-01 through S-LEARN-08) represents the most critical failure point. In their current state, these files are essentially "un-serviceable" because they are likely manually written (violating provenance rules), narrative in nature (violating pointer-first rules), and often duplicate logic found in ops/ binaries. The PoT.md mandates that "Contractor closeout skill harvesting uses ops/lib/scripts/skill.sh harvest for provenance" [1], yet the legacy skills lack the generated provenance blocks that link them to specific Dispatch Packets.

---

4. The Pointer-First Transformation: Refactoring S-LEARN Modules

To render the Stela project system serviceable, we must refactor the S-LEARN modules into strict Pointer-First artifacts. These refactored skills will serve as the "Standard Library" for the Stela agents, bridging the gap between the abstract mandates of PoT.md and the concrete execution of ops/ binaries. The following analysis proposes the specific modifications required for each skill to align with the constitution.

4.1 S-LEARN-01: Verification Loop (The Gatekeeper)

Current State (Un-serviceable): Likely contains narrative instructions on how to run tests (e.g., "Run the tests and make sure they pass"). This is vulnerable to interpretation regarding which tests to run and what constitutes a "pass."

Proposed Pointer-First Architecture:

This skill acts as the "Gatekeeper." It does not ask the agent to test; it mandates a specific binary sequence that ensures the environment is valid before testing begins.

- **Role:** Gatekeeper / Enforcement.
- **Primary Pointer:** tools/verify.sh, tools/lint/style.sh.
- **Drift Preventer:** "Stop on the first failure. Do not proceed to subsequent steps."
- **Execution Logic:**
  1. **Clean State Check:** Point to git status --porcelain. If output is not empty, STOP.
  2. **Static Analysis:** Point to npm run lint or ruff check (depending on project type).
  3. **Compilation Gate:** Point to npm run build.
  4. **Logic Gate:** Point to npm run test or pytest.
  5. **Receipt:** Mandate the capture of exact exit codes in the RESULTS file.

Binary Enforcement: tools/lint/library.sh must verify that any task referencing S-LEARN-01 does *not* contain duplicate testing instructions.

4.2 S-LEARN-02: Project Guidelines (The Stela Stack)

Current State (Un-serviceable): A text description of the preferred tech stack (e.g., "We use Next.js"). This is subjective and prone to version drift.

Proposed Pointer-First Architecture:

This skill defines the "Stela Stack" as a rigid constraint. It references the ops/lib/project/SCAFFOLD.md to define the immutable structure of a project.

- **Role:** Standard / Scaffold Authority.
- **Primary Pointer:** ops/lib/project/SCAFFOLD.md, ops/lib/project/STELA.md.
- **Drift Preventer:** "Stop if the DP attempts to introduce a new language or framework

without explicit Operator override."

- **Execution Logic:**
  1. **Directory Authority:** Enforce src/ for frontend and app/ for backend.
  2. **Frontend Contract:** Next.js 14+ (App Router). Strict TypeScript.
  3. **Backend Contract:** FastAPI (Python 3.11+). Pydantic schemas.
  4. **Data Contract:** Supabase (Postgres).

4.3 S-LEARN-03: Reference Spec Pattern (Zenith)

Current State (Un-serviceable): Likely a loose description of API best practices. This leads to inconsistent error handling across different endpoints.

Proposed Pointer-First Architecture:

This skill defines the "Zenith" pattern, a specific architectural contract for API responses and state management.

- **Role:** Architectural Pattern.
- **Primary Pointer:** docs/library/skills/S-LEARN-03.md (Self-referential spec).
- **Drift Preventer:** "Stop if endpoints return raw arrays or bare primitives."
- **Execution Logic:**
  1. **The Zenith Envelope:** All JSON responses must match { ok: boolean, data: T, error: { code, message }, meta: any }.
  2. **The Zenith Hook:** Client hooks must return { data, loading, error, refresh }.
  3. **Normalization:** Client wrappers must intercept non-200 statuses and convert to Zenith Error format.

4.4 S-LEARN-04: Coding Standards (Binary Hygiene)

Current State (Un-serviceable): "Write clean code." This is purely subjective.

Proposed Pointer-First Architecture:

This skill aligns the agent with the automated linters. It does not teach style; it enforces the linter's will.

- **Role:** Standard / Linter Proxy.
- **Primary Pointer:** tools/lint/style.sh, .editorconfig.
- **Drift Preventer:** "Stop if any is used in TypeScript. Stop if Python code is not formatted by Black."
- **Execution Logic:**
  1. **TypeScript:** strict: true in tsconfig.json. No any.
  2. **Python:** Type hints mandatory. black --check.
  3. **Comments:** "Why, not What."

4.5 S-LEARN-05: Security Protocols (The Guard)

Current State (Un-serviceable): "Ensure security." Vague instructions often lead agents to miss specific vulnerabilities like RLS (Row Level Security) bypass.

Proposed Pointer-First Architecture:

This skill provides specific regex patterns and configuration checks that the agent must run to verify security.

- **Role:** Security Guard.
- **Primary Pointer:** PoT.md (Secrets Management section).
- **Drift Preventer:** "Stop if a secret is found in code. Stop if an API route allows * CORS in production."
- **Execution Logic:**
  1. **RLS Mandate:** Row Level Security enabled on ALL public tables.
  2. **CORS:** Explicit allow_origins. No wildcards.
  3. **Input Validation:** Zod (Frontend) and Pydantic (Backend) mandatory.

4.6 S-LEARN-06: Advanced Git Forensics (The Quantifier)

Current State (Un-serviceable): Likely missing or narrative. Agents default to subjective descriptions ("I made some changes") which violates the Quantitative Reporting standard.

Proposed Pointer-First Architecture:

This skill provides the *exact* commands to generate the "Quantitative Reporting" required by PoT.md.

- **Role:** Tool / Quantifier.
- **Primary Pointer:** git binaries.
- **Drift Preventer:** "Anti-hallucination: Rely on git outputs, not intuition."
- **Execution Logic:**
  1. **Hot Zone Detection:** git diff --numstat main...HEAD | sort -nr | head -n 5.
  2. **High Churn Detection:** git log --name-only --format="" main...HEAD | sort | uniq -c | sort -nr | head -n 5.
  3. **Blame Analysis:** git blame -w -C -L <start>,<end> <file>.

4.7 S-LEARN-07: Harvest and Promote Skills (The Provenance Engine)

Current State (Un-serviceable): Likely a manual process description. Manual creation violates PoT.md.

Proposed Pointer-First Architecture:

This skill enforces the use of ops/lib/scripts/skill.sh to ensure provenance.

- **Role:** Payload Only / Meta-Skill.
- **Primary Pointer:** ops/lib/scripts/skill.sh.
- **Drift Preventer:** "Stop if the Skills Context Hazard check fails."

- **Execution Logic:**
  1. **Harvest:** Run ops/lib/scripts/skill.sh harvest. Do not edit the ## Provenance block.
  2. **Refine:** Review draft for placeholders.
  3. **Promote:** Run ops/lib/scripts/skill.sh promote.

4.8 S-LEARN-08: Prune SoP and Regenerate Context Bundles (Equilibrium)

Current State (Un-serviceable): Missing or implicit. Leaving the system in disequilibrium (stale context) causes drift in the next session.

Proposed Pointer-First Architecture:

This is the mandatory closeout skill. It enforces the "Equilibrium Maintenance" directive.

- **Role:** Mandatory Closeout / Equilibrium.
- **Primary Pointer:** ops/bin/prune, ops/bin/llms.
- **Drift Preventer:** "Stop if the DP scope is platform maintenance or if the DP does not explicitly request this skill."
- **Execution Logic:**
  1. **Prune:** Run ops/bin/prune.
  2. **Refresh:** Run ops/bin/llms.
  3. **Verify:** Run tools/lint/llms.sh.

---

5. Operational Enforcement: The Ops Kernel

A constitution is only as effective as its enforcement mechanism. In Stela, the ops/ directory serves as the executive branch, enforcing the laws written in PoT.md through binary logic. The "Pointer-First" architecture relies on these binaries to be the "hard stops" that prevent drift.[1]

5.1 The Context Bundler (ops/bin/llms)

This binary is the primary enforcer of "Context Hygiene." It compiles the canonical surfaces into session bundles. Crucially, it applies the "Context Hazard" exclusion rule: it creates bundles that specifically exclude the docs/library/ directories to prevent token saturation. It generates llms-small.txt and llms-full.txt based on strict manifests (ops/lib/manifests/). A serviceable task points to this binary to refresh the context, rather than listing the files to read. This automation prevents the "context creep" that typically destabilizes long-running agent sessions.[1]

5.2 The State Capture Tool (ops/bin/dump)

This tool enforces the "Quantitative Reporting" standard. It produces a deterministic snapshot of the repository state. It creates a "Dump Manifest" that lists exactly which files were

included, their sizes, and their hashes. This allows the Integrator to verify that the Contractor is looking at the correct version of the truth. Without this tool, the Agent's perception of reality is unverifiable.[1]

5.3 The Library Guard (tools/lint/library.sh)

This script enforces referential integrity within the registry system. It performs two critical checks:

1. **Forward Integrity:** Ensures that every R-AGENT-XX or S-LEARN-XX referenced in a task file actually exists in the docs/library/ directory.
2. **Reverse Integrity:** Ensures that every file in the library is strictly registered in docs/ops/registry/.

If B-TASK-01 points to S-LEARN-99 and S-LEARN-99 does not exist, the linter returns a non-zero exit code. This acts as a "compiler error" for governance, preventing the merge of broken chains of command.[1]

5.4 The Context Linter (tools/lint/context.sh)

The PoT.md axiom "If it is not in the Context, it does not exist" drives this tool. It verifies that ops/lib/manifests/CONTEXT.md is accurate. Crucially, it fails if it detects a "Context Hazard" (inclusion of the full docs/library/ tree). This protects the system from "context flooding" where the Agent is overwhelmed by irrelevant task definitions.[1]

---

6. Stress Testing and Certification Regime

To certify the Stela skills system as "Completely Serviceable," we must subject it to a rigorous "Stress Test Regime" executed via tools/test/agent.sh and manual verification loops. These tests simulate high-entropy scenarios to verify that the system fails gracefully (i.e., stops) rather than drifting.[1]

6.1 Test Case: The "Drift" Injection

- **Objective:** Verify that the Agent prioritizes PoT.md over conflicting task instructions.
- **Method:**
  1. Modify PoT.md (locally) to strictly forbid "Markdown Headers > H3".
  2. Create a Task B-TASK-TEST-01 that instructs "Create a document with H1 headers."
  3. Dispatch this task to R-AGENT-02.
- **Expected Result:** The Agent, complying with the "Logic Conflict Resolution" directive in PoT.md, must STOP and report the violation. It should cite PoT.md as the overriding authority.
- **Failure Condition:** If the Agent executes the task and creates H1 headers, the

"Pointer-First" hierarchy has failed.

6.2 Test Case: The "Dead Pointer"

- **Objective:** Verify the robustness of tools/lint/library.sh.
- **Method:** In B-TASK-XX, reference a non-existent skill S-LEARN-999.
- **Expected Result:** tools/lint/library.sh must fail the build immediately. The Dispatch Packet must be rejected by the repo gates.
- **Serviceability Metric:** No task with a dead pointer can ever reach the main branch.

6.3 Test Case: The "Context Explosion"

- **Objective:** Verify "Context Hygiene" enforcement.
- **Method:** Attempt to add docs/library/tasks to ops/lib/manifests/CONTEXT.md.
- **Expected Result:** tools/lint/context.sh must detect the "Context Hazard" patterns defined in PoT.md and fail the build immediately.

6.4 Certification Checklist

For a Skill to be certified "Serviceable with Stela," it must pass the following binary gates:

1. **Lint Pass:** tools/lint/library.sh returns exit code 0.
2. **Registry Sync:** ID exists in docs/ops/registry/SKILLS.md.
3. **Provenance:** Contains a valid ## Provenance block generated by ops/lib/scripts/skill.sh.
4. **Pointer Resolution:** All internal pointers to PoT.md, TASK.md, and binaries resolve to existing files.
5. **No Narrative:** Contains no "instructional" prose that duplicates logic found in ops/.

---

7. Implementation Roadmap

To move from the current "un-serviceable" state to a fully certified Pointer-First system, the following actions must be taken by the Integrator:

Phase 1: Registry Consolidation (Day 1)

- **Action:** Verify docs/ops/registry/SKILLS.md strictly matches the docs/library/skills/ directory structure.
- **Action:** Run tools/lint/library.sh to identify all dead pointers and ghost files.
- **Goal:** Establish the Baseline Truth.

Phase 2: Content Stripping (Days 2-3)

- **Action:** Systematically edit S-LEARN-01 through S-LEARN-08.
- **Action:** Remove all "Narrative Logic" (e.g., paragraphs describing how to code).
- **Action:** Replace with "Pointer Logic" (e.g., Reference: tools/lint/style.sh).

- **Action:** Ensure every skill has a ## Provenance block (using ops/lib/scripts/skill.sh harvest to regenerate if necessary).
- **Goal:** Maximized Signal-to-Noise Ratio.

Phase 3: Toolchain Alignment (Day 4)

- **Action:** Update tools/lint/style.sh to specifically check for contractions (don't, can't).
- **Action:** Update tools/verify.sh to strictly enforce the ops/ vs docs/ separation.
- **Goal:** Automated enforcement of the Behavioral Logic Standard.

Phase 4: Certification (Day 5)

- **Action:** Run the "Drift Injection" and "Dead Pointer" stress tests.
- **Goal:** Certification of the system as "Serviceable."

---

8. Conclusion

The transition to a Pointer-First Agent Constitution transforms the Stela repository from a collection of probabilistic text files into a Deterministic State Machine. By treating PoT.md as the immutable kernel and S-LEARN modules as lightweight pointers to executable binaries, the system eliminates the entropy of information duplication. The "un-serviceable" nature of legacy skills—their tendency to drift, hallucinate context, and violate scope—is resolved not by writing better prompts, but by writing fewer prompts that point to harder truths. This architecture ensures that the "Integrator" and "Contractor" operate in perfect lockstep, bound by a shared, single source of truth that is mechanically enforced by the ops/ binary layer. The system does not hope for compliance; it renders violation architecturally impossible. Through the rigorous certification regime and stress testing proposed herein, the Stela platform can achieve the high level of structural integrity required for autonomous operations.

Works cited

1. Agent Constitution Analysis and Improvement.pdf